Q1. What was guided during the project-related sessions.

Answer 1 :

During the project-related sessions, we were guided through the complete development of a full-stack web application, covering both theoretical concepts and practical implementation.

For backend development, we used **Spring Boot (latest version 4.0)** to build RESTful APIs. We learned how to design and implement endpoints for CRUD operations (Create, Read, Update, Delete). The architecture included controller, service, and repository layers, along with dependency injection for better code organization.

A key focus area was data validation, where backend validation was implemented, these ensured that only valid and properly formatted data is processed before storing it in the database. We also handled validation errors using exception handling to return meaningful responses.

For the frontend, we used **React** to build dynamic and responsive user interfaces. We learned how to integrate frontend components with backend APIs using HTTP requests and display real-time data. Client-side validation was also implemented to improve user experience.

For data storage, we used **MySQL**, where we designed tables, defined relationships, and performed SQL operations. The backend was connected using ORM tools like JPA/Hibernate for efficient data management.

Additionally, a **live project** titled "**Ecommerce Web Application**" was developed and demonstrated using all the above technologies. This helped us understand real-world application flow, including API integration, validation, and UI rendering. The complete project source code was also shared via a **GitHub repository**, allowing us to practice and explore the implementation independently.

Ecommerce Web Application Architecture using Spring Boot and React

---

Technology Stack

1 Backend — Spring Boot

- Spring Boot – To build RESTful APIs
- Spring Security + JWT – Authentication and authorization
- Spring Data JPA (Hibernate) – ORM layer for database access
- MySQL / PostgreSQL – Relational database
- Maven / Gradle – Build and dependency management
- Hibernate Validator – Input validation
- JUnit 5, Mockito – Testing framework
- Swagger / OpenAPI – API documentation

---

2 Frontend — React

- React – Component-based UI framework
- React Router – For navigation
- Axios / Fetch API – For API calls
- useState and useEffect – State management (basic)
- Bootstrap / Material-UI / Tailwind – Styling
- Vite / CRA – Development and builds

---

3 Database — MySQL (Using ORM)

- MySQL – Used as the primary relational database
- Hibernate (via JPA) – Handles ORM (Object Relational Mapping)
- Entity Classes – Represent database tables
- Repository Interfaces – Perform CRUD operations without writing SQL
- Automatic Table Mapping – Java objects mapped to database tables
- Query Methods / JPQL – For custom data retrieval
- Database Relationships – One-to-One, One-to-Many, Many-to-Many mappings
- Transaction Management – Ensures data consistency and integrity

Q2. The problem statements shared.

Answer :

Design and develop a scalable and efficient **full-stack web application** using **Spring Boot**, **React**, and **MySQL** to manage core business operations such as product management, customer handling, order processing, and billing.

The system should provide a seamless user experience by enabling users to perform operations like registration, login, product browsing, and order placement through a responsive frontend interface. The backend should expose secure and well-structured RESTful APIs to handle business logic, data processing, and communication with the database.

The application must ensure proper **data validation**, **error handling**, and **role-based access control**, while maintaining a layered architecture consisting of controller, service, repository, and entity layers. It should support CRUD operations, efficient data retrieval, and real-time interaction between frontend and backend using JSON.

The project should incorporate robust frontend features using **React**, including a component-based architecture for reusability, efficient state management using hooks like useState and useEffect, and seamless navigation using React Router. It should support dynamic data rendering by integrating with backend APIs through Axios or Fetch, along with responsive UI design using frameworks like Bootstrap or Tailwind CSS. Additionally, the application should include client-side validation, user-friendly error handling, and optimized performance through proper component structuring and lazy loading where applicable.

Additionally, the system should be designed with modularity and scalability in mind, incorporating features such as authentication (JWT), search and filtering, and integration capabilities. The project should follow best practices in software development, including testing, version control using **GitHub**, and structured deployment.

The final solution should demonstrate a complete end-to-end workflow—from user interaction on the frontend to data persistence in the database—ensuring reliability, performance, and maintainability.

Q3. The steps/process explained to complete the project

Answer

The following step-by-step process was explained to complete the full-stack web application using **Spring Boot**, **React**, and **MySQL**:

🔹 1. Requirement Analysis

- Understand the problem statement and project scope
- Identify features (e.g., login, CRUD operations, reports)
- Define user roles and system functionality

🔹 2. Database Design

- Design tables and schema in MySQL
- Define relationships (One-to-One, One-to-Many, Many-to-Many)
- Identify primary keys and foreign keys

🔹 3. Backend Development

- Create Spring Boot project
- Define Entity classes (mapping with database tables)
- Create Repository interfaces using JPA
- Implement Service layer for business logic
- Develop Controller layer for REST APIs
- Implement validation using annotations (@NotNull, @Email, etc.)

🔹 4. API Development & Testing

- Create CRUD APIs (GET, POST, PUT, DELETE)
- Test APIs using tools like Postman
- Handle exceptions and return proper responses

🔹 5. Frontend Development

- Create React project structure
- Design UI using components
- Implement routing using React Router
- Fetch data from backend using Axios/Fetch
- Manage state using useState and useEffect

🔹 6. Integration (Frontend + Backend)

- Connect React frontend with Spring Boot APIs
- Handle JSON data exchange
- Display dynamic data on UI

🔹 7. Validation & Error Handling

- Implement backend validation (JSR-303)
- Add frontend form validation
- Display user-friendly error messages

🔹 8. Testing

- Perform unit testing (JUnit, Mockito)
- Check all functionalities (CRUD, login, validation)
- Fix bugs and optimize performance

🔹 9. Deployment / Execution

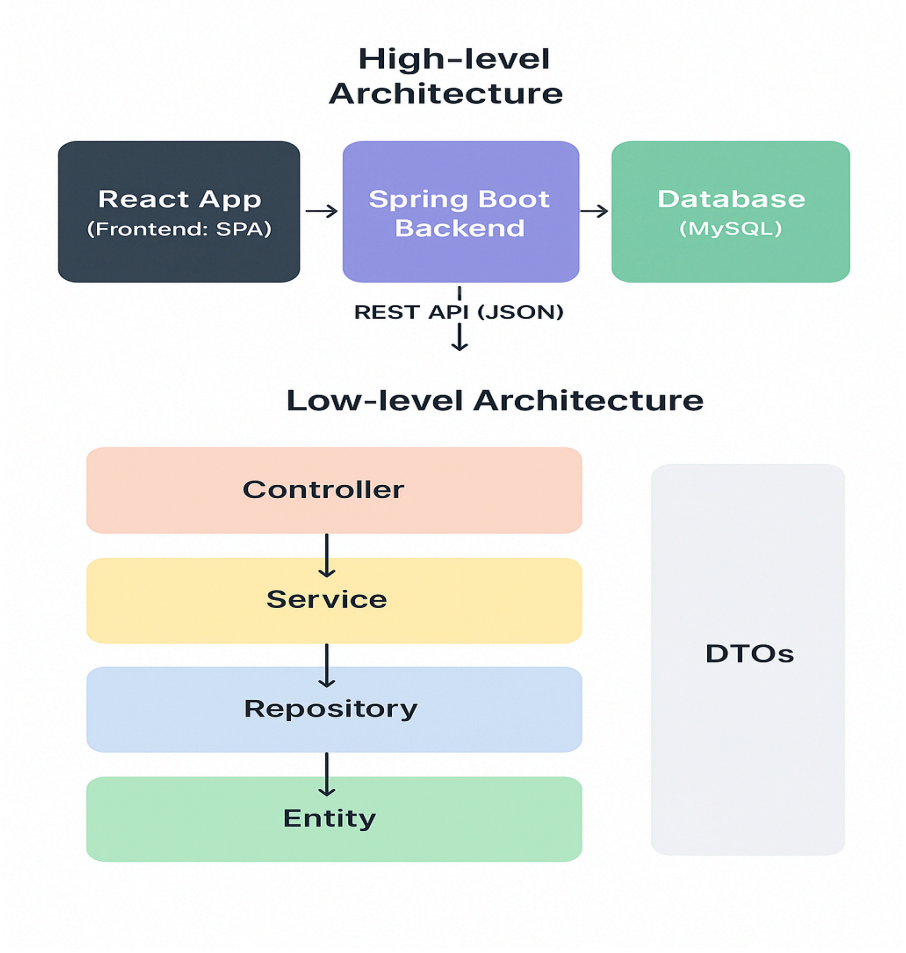
- Run backend server and frontend application
- Verify end-to-end functionality
- Prepare project demo

🔹 10. Version Control & Sharing

- Upload project to **GitHub**
- Maintain code versions and updates
- Share repository for collaboration and review

High-Level Architecture

Frontend (React) communicates with the Backend (Spring Boot) via REST APIs (JSON). Backend interacts with the Database (MySQL/PostgreSQL).



Low-Level Architecture (Backend Layered)

- Controller Layer – Handles HTTP requests and responses.
- Service Layer – Contains business logic and validations.
- Repository Layer – Manages database operations using JPA.
- Entity Layer – Represents database tables as Java classes.
- DTOs – Used for data transfer between layers.

React Folder Structure

```
retail-app/
|
```

```
├── src/
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── ProductList.jsx
│   │   ├── ProductCard.jsx
│   │   └── Cart.jsx
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── ProductDetails.jsx
│   │   ├── Checkout.jsx
│   │   └── Login.jsx
│   ├── services/
│   │   └── productService.js
│   ├── App.js
│   └── index.js
│
└── package.json
```

Sample Data Flow (Get Product by ID)

- User clicks product → React navigates to /product/{id}.
- React calls backend API via Axios.
- Spring Boot Controller receives the request → Service → Repository.
- Repository fetches data from DB and returns JSON.
- React displays the product details in UI.

Optional Advanced Features

- Authentication with JWT
- Search and Filter with Query Parameters
- Caching (Spring Cache / Redis)
- Logging with SLF4J + Logback

7Example Use-Case Modules

- Product Management – Product, Category
- Customer Management – Customer, Address
- Orders & Billing – Order, OrderItem, Invoice
- Inventory – Stock, Supplier
- Authentication – Form based authentication/ Dao Authentication /JWT
- Authorization – Role based authorization – USER/ADMIN

Q4 : Sample Project Topics shared with students
Answer :

Project Topics

- Retail Billing
- Smart Queue Management System
- Emergency Home Service
- Turf Management
- Bookstore Application
- Mental Health Early Warning
- Financial Crime Detection
- PMC Civic Connect
- Multi-Vendor E-commerce Platform
- Multi-Business Application
- Finance Management
- Car Rental Billing
- Smart Recipe
- Retail Billing
- Exam Portal
- LMS (Learning Management System)
- ATS (Applicant Tracking System)
- Used Car Auction
- Pharmacy Billing System
- Online Food Delivery Application